The Economic Constitution of Possibility: Technical and Conceptual Supplement

How to use this supplement

Jian Gao (高健)

Independent researcher · innersummer@hotmail.com · ORCID 0009-0009-5002-1655

This supplement gives self-contained definitions, conditions, decisive proofs, and source correspondences for the results used in the main paper. Appendix A covers society, exit, clearing, and renewal; B covers capital, innovation, and public production; C covers public functions and independent planning; D specifies new design contracts for money and bounded audit. Each original construction retains its own types, occurrences, and resource units. Their joint argument does not require them to constitute a single actual execution.

In the formal sources, a “payment” is a complete object supported by an original occurrence that actually disposes of responsibility and produces a successor. A cash transfer is one possible component; service, maintenance, performance, responsibility transport, and the actual next state must also be supplied as required by the original contract. “Source” identifies the occurrence from which a standing or responsibility arises. A “consumer” is a subsequent declaration that actually uses a result and preserves its content. The reasons for the paper's normative principles are developed in the main text.

Source identities, original certification records, and current public targets are registered separately in this paper's source correspondence and availability notes. The written proofs reuse established constructions and preserve the quantifiers of the original results.

Appendix A. Complete Action, Clearing, and Renewal

A.1. What makes an exit path actionable [S1]

Fix an actual payment p of the original program. Let C be the finite set of future codes generated by the original law, let r be a relation for independent continuation distinct from the original employment relation, and let $J = \mathrm{Fin}(|\mathrm{served}|)$ index the slots actually served in this payment. The original employment relation's working future has been withdrawn while the worker remains. Continuation outside the role is carried by r.

For $c \in C$, define liveness by three conditions: c continues on the target side of r, c is not idle, and c is not captured by the original control relation. A feasible edge (c, j) also carries two witnesses: the incidence of payment p with c, and a route from c to the jth actually served traversal. The service index maps to the correspondingly indexed event of the original execution and preserves that event's service receipt. Action, payment, route, and service are thereby joined in the same object.

Proposition A.1 (the complete characterization of exit). At every original finite visit depth, the actual-exit type is inhabited if and only if there exist $c \in C$ and $j \in J$ jointly satisfying continuation outside the role, non-idleness, non-capture, payment incidence, and route incidence. The maximum matching size K of the original finite graph satisfies

$$K > 0 \quad \Longleftrightarrow \quad \text{the actual-exit type is inhabited.} \tag{A.1}$$

Proof. Project the code, service slot, and edge from an actual exit. Expanding the code's liveness yields the first three conditions; reading the edge's payment and route fields yields the last two. Conversely, assemble the first three conditions into a live code, then use the two witnesses at the same c, j to construct a feasible edge. Transport from the original service index to the actual event supplies the actual service receipt, completing the exit.

A nonempty finite matching contains an edge, and a single edge defines a singleton matching. Thus positive matching size is equivalent to the existence of an edge and hence to the preceding characterization. Neither endpoint repeats in a matching, so

$$K \le |C_{\mathrm{live}}|, \qquad K \le |\mathrm{served}| \le \mathrm{capacity}(p).$$

The original canonical source has three requests and two actually served slots. Its only live actionable exit code is leave, and it has an actual edge. The bounds therefore give $K = 1$, while the unserved queue remains nonempty. ■

The source also provides three constructions with zero capacity. They respectively remove non-capture, payment incidence, or route incidence while retaining the other materials stated by each construction. The first has no live exit code. In either of the latter two, any feasible edge would supply a witness of a type that has been made empty. All three therefore have zero capacity, although they belong to distinct source branches. If capacity admitted a left inverse recovering the source, all three would be recovered as $g(0)$ and thus identified, a contradiction.

Removing payment incidence here preserves the underlying payment event: what is absent is that payment's incidence with this exit action. Likewise, an actual service may exist without a route from the action to that service. These independently necessary links explain the use of equation (1) in the main text.

A.2. Different complete portfolios at the same positive capacity [S2]

Fix any depth d. The original domain family D has two constructors: rights retention and health. Each domain's bearer, relation, and stage come from its own relation seed. The complete source takes prefix cut 3 of the same original queue, with actual capacity 2 and served positions 0 and 1. Rights retention selects the first position; health selects the middle position.

For domain $a \in D$, a service incidence contains a served index j, an equality between the unit at that slot and the original generated unit for a, and the actual receipt at j from the same decrementing execution. Combining this with relational benefit, the target role, payment and route incidence, and a non-captured edge yields $\mathcal{A}_d(a)$. Define the complete action portfolio as the dependent product

$$\mathcal{P}_d = \prod_{a \in D} \mathcal{A}_d(a). \tag{A.2}$$

It requires actual supporting material for every installed domain.

Proposition A.2 (a positive scalar does not decide completeness). The original complete source x_d and the single-service source y_d satisfy

$$K(x_d) = K(y_d) = 1, \qquad \mathcal{P}(x_d) \neq \emptyset, \qquad \mathcal{P}(y_d) = \emptyset. \tag{A.3}$$

There is no $q : \mathbb{N} \to \mathrm{Prop}$ that characterizes portfolio inhabitation by an equivalence on both sources.

Proof. In x_d, take slots 0 and 1 with their original service receipts for the two domains. Join each to the same leave action as a feasible edge to obtain the dependent product. Slot distinctness follows from the distinct positions in the original queue; the first projections of the two actual service identities are respectively 0 and 1.

The source y_d takes cut 1 of the same queue and executes only the first actual maintenance service. The original root event, payment occurrence, payment rule, and route rule are preserved, but the served list contains only slot 0. Rights retention still has an actual edge. If health had a service incidence, the unique slot's unit would have to equal the unit at the original middle position, contradicting their distinctness. Its portfolio is therefore empty. Each source has exactly one live action and at least one edge, so both capacities equal 1.

If q existed, the complete source would give $q(1)$. The reverse direction of the equivalence for the single-service source would then turn $q(1)$ into an impossible complete portfolio. ■

The original consumer also retains the exact counterexamples obtained by deleting payment incidence, route incidence, non-capture, or actual service. The successor interface connects the portfolio at $d + 1$ to the original actual payment after repair, economic root event, folded image, and landing point. The installed domain family is the explicit scope of the proposition. Institutional applications must provide service and budget for each additional domain.

A.3. Whole-domain readouts, repair obstructions, and source refinement [S3]

Let P_d^c be the proposition witnessed by the original captured source, and P_d^r the proposition witnessed by the original non-captured source with a broken route. The source type is

$$\Omega_d = P_d^c \sqcup P_d^r.$$

Both branches have original actual constructions, denoted c_d, r_d, and their constructors are distinct. For all installed domains $\mathcal{D}$ of the original program, define

$$\rho_d(x) = (K_d(x), \{D \in \mathcal{D} : \text{the domain has a feasible edge}\}). \tag{A.4}$$

Every candidate edge in the captured branch contradicts non-capture. Every edge in the broken-route branch would have to supply a route witness excluded by that source. Hence

$$\rho_d(c_d) = \rho_d(r_d) = (0, \emptyset).$$

The original repair type has two elements: release capture and restore the route. The repair compiler, fixed before emission, already has the failure-kind map ϕ_d and rule R_d. Define the original repair authority by

$$\kappa_d(x) = R_d(\phi_d(x)).$$

Here $\kappa_d(c_d)$ releases capture and $\kappa_d(r_d)$ restores the route. Preserving the respective unaffected incidences, the original geometry gives:

Source	Capacity after releasing capture	Capacity after restoring the route
Captured	1	0
Broken route	0	1

The effective repair is therefore exactly $\kappa_d(x)$.

Proposition A.3 (no post-processing of a coarse readout recovers repair). For any output type Y and readout $q = f \circ \rho_d$, there is no $g : Y \to \mathcal{R}$ satisfying $g(q(x)) = \kappa_d(x)$ for every x. The refined readout

$$\hat{\rho}_d(x) = (\rho_d(x), \phi_d(x)) \tag{A.5}$$

is injective on the entire original source type and correctly generates the unique effective repair.

Proof. If g existed, the equal values of ρ_d would remain equal after applying f and g. This would give $\kappa_d(c_d) = \kappa_d(r_d)$, contradicting the distinctness of the two repairs. The argument imposes no finiteness, countability, or numerical requirement on Y.

If two refined values are equal, their failure kinds agree, so their sources lie in the same branch of the sum type. Within that branch, each carries only a proof of the same proposition; proof irrelevance gives equality of the sources. The refined repair readout $(y, k) \mapsto R_d(k)$, applied to (A.5), returns $\kappa_d(x)$. The table proves its effectiveness and uniqueness. ■

The source also admits an automorphism σ that exchanges the two branches. It preserves ρ_d while exchanging the two repairs, and the repair exchange has no fixed point. If a right section over

the visible readout were natural under this symmetry, the selected preimage of the fixed visible value would also have to be fixed. Applying κ_d would then produce a fixed point of a fixed-point-free exchange, a contradiction. This gives the corresponding obstruction to natural reconstruction of authority.

The positive result first supplies proof-carrying repair in the specified geometry. Actual support is separately provided by the original installed branch: two occurrences have the same root and compatible complete writebacks and successors, while their original pre-emission failure fields select different repairs. Every compared field retains its dependent type; the two sources are not asserted to agree in all respects.

A.4. Exit through execution, service, and a later continuation [S4]

Fix an original old visit o, source index a, and post-settlement starting point i. The original standing continuation L_i generates the person's actual exit X_i, continuation outside the role C_i, and disposition of external responsibility D_i. Write the three payments as

$$p_0 = L_i.\text{sourcePayment}, \qquad p_1 = C_i.\text{executedPayment}, \qquad p_2 = C_i.\text{successorPayment}.$$

Four occurrences e_0, e_1, e_2, e_3 are actually produced by this chain. The original interface identifies the standing continuation's target payment with p_1, the outside-role continuation's source and target payments with p_1, p_2, and the following continuation's source payment with p_2. Thus

$$p_0.\text{next} \simeq e_1, \qquad p_1.\text{next} \simeq e_2, \qquad p_2.\text{next} \simeq e_3. \tag{A.6}$$

The symbol $\simeq$ denotes dependent equality after transport along proved index equalities, preserving the distinct occurrence identities of source and target.

Let $R_i(c)$ be the complete temporal contract: equation (A.6); incidence of the target response with the person's exit; employment at the source and release at the target; continuation of the same c in an independent relation without capture; disposition of external responsibility at e_2 using its own payment and writeback; actual paid consumers for every original service channel at later occurrences; and injective authority and service-occurrence maps for the four stages. Let $S_i(c, j)$ be the complete feasible-edge condition of A.1 at this actual service depth.

Proposition A.4 (reconstruction of the complete lifecycle in both directions). The original complete exit-lifecycle type M_i is inhabited if and only if

$$\exists c, \exists j, \quad S_i(c, j) \wedge R_i(c). \tag{A.7}$$

The original program generates such a joint witness for every o, a, i.

Proof. The data of M_i consist of an actual exit continuation and a temporal contract indexed by its code. Projecting its code, slot, edge, and temporal fields yields the right-hand side. Conversely, use S_i to construct a complete feasible edge and pair it with R_i at the same c to obtain M_i. This direction does not invoke inhabitation of the completed object. The original positive source generates the edge of A.1 with code leave. Transport the original temporal contract along the equality with C_i's code to obtain the joint witness. ■

The action readout at all four stages may be leave even though the occurrences differ. A function recovering authority solely from the action would identify stages 0 and 1. The original structural response separately reads p_1's own action, payment, service, and successor and preserves non-capture. Four policy, service, administrative, and employment retypings that retain these materials but assert capture of the original response by the original geometry are empty by direct contradiction. The actual full-capture comparison retains its own different geometry and has zero capacity.

At e_2, the local open-responsibility type may be empty while the same payment supplies an external-responsibility instance and an independent root consumer. If local emptiness implied external emptiness, applying this implication to the original external instance would give a contradiction. Local completion therefore preserves consequences across sources.

The original rights-retention and mandate relations each have a non-employment exit interface. In a mandate, the second action consumes precisely the predecessor's actual post-exit successor: the original authorization first ends, and the new controller is generated by adding one to the maximum identifier in the original history. New obligations connect to the original service slots, old responsibilities are transported and preserved, and the terminated authorization remains closed. These concrete interfaces support extending exit tests to different relations. Each added relation requires its own bearers, routes, and service contract.

A.5. Joint clearing of every transaction leg [S5]

Let I be a finite set of transaction legs, and let $\sigma : I \to I$ be the original counterparty map with $\sigma^2 = \mathrm{id}$. Every i supplies a bearer a_i, responsibility q_i, and actual slot j_i. The original transaction preserves $a_i \neq a_{\sigma(i)}$ and $j_i \neq j_{\sigma(i)}$, and the map from legs to slots is injective. A duplicate-free exhaustive enumeration of the original legs generates an executable performance event u_i with exact incidence to each leg's original responsibility.

Proposition A.5 (complete performance and an uncleared counterparty). Executing all original legs in order leaves no open settlement in the final state. Executing u_i alone cannot perform counterparty leg $\sigma(i)$.

Proof. Induct on prefixes of the enumeration. When u_i is added, its original responsibility incidence and actual performance receipt write j_i as a fulfilled terminal slot. Updates to distinct slots preserve earlier terminal slots, while all other slots retain their original values. Every leg in the processed prefix is therefore fulfilled. By exhaustiveness, every original leg is terminal in the final state. Any open settlement would have to arise from a live slot of an original leg, contradicting the distinct terminal constructor.

If only u_i is executed, the inequality $j_i \neq j_{\sigma(i)}$ ensures that updating j_i leaves the counterparty slot unchanged and still live. ■

The clearing seed of the original four-leg instance is generated internally by the established complete execution. Once installed in the cycle root, legislative choices, mandates, operational authority, and whole-ledger writebacks at each depth connect through the installation coordinates; the actual successor is the next visit. Payment occurrences at different depths are distinguished by visit injectivity. The proof requires no pre-existing ledger of an already completed infinite execution.

In the actual partially cleared state, the original a leg is fulfilled and the b leg remains open. The credit seed is generated from b's open responsibility, includes a boundary source event and counterparty acceptance, and reads its risk bearer from that same responsibility. A later performance event is appended to the actual history and clears b's original slot. The original root request and debt lineage are preserved. In the fixed instance the progress budget is $2 \to 2$. This credit construction preserves provenance and actual settlement; write-downs and risk sharing require their corresponding new updates.

A.6. Preservation for every original payment, a strict frontier, and cofinal renewal [S6]

Fix an original registered cycle

$$C : \mathrm{CycleAt}(v, s, b), \qquad i_d = b + d + 2.$$

At index k, the actual current value is $c_k = r_0(s) + k$, the root event is ω_k, the complete current object is T_k, the native payment is $\bar{p}_k$, and the complete target birth is γ_k. The original payment type P_k is indexed by the fixed source, event, and actual exit and contains native writing, a complete responsibility successor, the person's exercise and response, exit, continuation outside the role, and external responsibility.

In this precise fiber, the responsibility successor has at most one element. Any $p, p' \in \mathsf{P}_k$ therefore have the same successor-target projection. Taking $p' = \bar{p}_k$ and using $c_k + 1 = c_{k+1}$ gives

$$\mathrm{NextPaidBy}(p) \simeq \omega_{k+1}. \tag{A.8}$$

This equality compares successor projections while preserving the distinctions between whole payments and adjacent occurrences.

Proposition A.6 (complete preservation and actual novelty). For every original C, observation parameter, depth d, and $p \in \mathsf{P}_{i_d}$, there is a complete payment step retaining input p whose actual target is T_{i_d+1}. It preserves the target's complete exit constitution, every installed channel, external responsibility, and its own native payment. The associated paid frontier strictly expands, and its strict order of complete constitutions coincides exactly with strict depth order.

Proof. Take the source payment to be p, the source whole ledger to be the original writeback of ω_{i_d}, and the target to be T_{i_d+1} under the original generating rule. Equation (A.8) connects the input payment's actual target to that object. New target services are fulfilled by $\bar{p}_{i_d+1}$ itself; the original generation provides the complete lifecycle contract of A.4 at the target. For each installed channel, read the target relation seed, original bearer and relation, actual service slot, and payment and route incidences to obtain an independent consumer. External responsibility and its consumer are likewise read from the target's actual exit.

The original recursion gives $i_{d+1} = i_d + 1$ and identifies the next cycle with this step's actual output. The target is therefore the source of the next lifecycle. All components use the same input, root, and actual successor and can be assembled jointly.

For strictness, let $K_k = \kappa(\gamma_k)$ denote complete service–exit authority, rather than numerical capacity. If $K_j = K_k$, projecting the payment occurrence and applying original visit injectivity

gives $c_j = c_k$; canceling the common starting value yields $j = k$. Define

$$H(a,m) = [\gamma_a, \dots, \gamma_{a+m-1}], \qquad A(a,m) = \{K_{a+r} : r < m\}.$$

The lists satisfy

$$H(a, m+1) = H(a,m) +\!\!+ [\gamma_{a+m}].$$

The authority K_{a+m} belongs to the new image. If it belonged to the old image, there would be $r < m$ with $K_{a+m} = K_{a+r}$; injectivity would force $m = r$, a contradiction. Consequently,

$$A(a,m) \subsetneq A(a,n) \quad \Longleftrightarrow \quad m < n. \tag{A.9}$$

Define $\prec_{\text{can}}$ by the complete exit constitutions at both endpoints and strict inclusion of their paid authority images. Both endpoint constitutions have already been generated. Substituting $i_d = b + d + 2$ and (A.9) gives

$$C_r \prec_{\text{can}} C_t \quad \Longleftrightarrow \quad r < t.$$

The new authority lies at input p's actual target: the left and right replays of the same original edge are indexed by i_d and $i_d + 1$, and (A.8) identifies the right-hand side. The old whole ledger, external consumers, and every channel identity are preserved along the original interfaces. Strictness and complete preservation therefore belong to the same payment step. ■

The recursive entry of the same cycle is always obtained from the preceding output. Each pulse jointly produces actual law revision and its target future, while the root law's current point, target, and own payment retain their actual indices. Given n, h, take $m = n + 1$. The original pulse-by-pulse construction gives a joint pulse at this same m, so

$$\forall n, h\ \exists m > n : \text{actual revision, the target future, and the complete root law hold jointly.}$$

Frontier growth records actual paid occurrences; instantaneous capacity can remain 1 throughout. The universal domain is the original registered cycles, payment types, and installed channels. Additional economic domains require their corresponding content from their own sources.

A.7. Conservative successors of an exhausted complete ledger [S7]

Let L be the complete responsibility rows of the exact source event, each with a natural-number progress budget $B(a)$. The entry rows L_T^0 of a native sequential action target are inherited through $i : L \to L_T^0$, with complete coverage and $B(i(a)) \le B(a)$. Every row $e \in L_T^1$ of the target's first writeback in turn has a native source $a' \in L_T^0$ and a lawful evolution satisfying $B(e) \le B(a')$.

Proposition A.7 (two-layer closure at zero budget). If $\forall a \in L,\ B(a) = 0$, then every such target has zero budget both at entry and at its first writeback. Its type of positive-budget rows is empty.

Proof. Take any $a' \in L_T^0$. Complete coverage supplies a with $i(a) = a'$. Hence $0 \le B(a') \le B(a) = 0$, so $B(a') = 0$. Now take $e \in L_T^1$. Its native source gives $0 \le B(e) \le B(a') = 0$, so $B(e) = 0$. A positive-budget row would satisfy both $B(e) = 0$ and $B(e) > 0$, a contradiction. ■

The original actual instance first consumes contribution capacity from 1 to 0. A same-source revision then generates exactly three complete live rows: inherited contribution, inherited obstruction, and a new residual. All three have zero budget. The new residual still has its own actual answer and independent handling role. For every original visit and revised root visit, this complete classification holds together with the universal closure above. An answer can therefore exist while positive budget is absent. New provision or admission must supply its own source and actual update to change these conditions.

Appendix B. The limits of contribution claims and the continuation of production

What conditions support the elimination of capital relations, the payment of real contributions, and the continuation of public production? This appendix gives, in order, an exact characterization of permission support, a renewal gate for paid innovation, a cumulative receipt bound for an original issuance source, and actual production through method reuse and after voluntary exit. Each result specifies how the same resources, responsibilities, and occurrences enter subsequent action.

All source locations are fixed at commit **df421ff600085cbc290a7ae946193961bed7c83b**, denoted by S. [CF1]–[CF5] and [PP1]–[PP4] are provenance codes for the original declarations and their direct uses. Three directory prefixes shorten the locations: A =`Lean/SaturationMonoid/ProcessGame/Society/Institutions/Capitalism/Universal/Abolition/Accounting/`; I =`Innovation/` under the same `Abolition/` directory; and P =`Lean/SaturationMonoid/ProcessGame/Society/PublicFunction/Productivity/`. Declarations are listed under their original names.

B.1. Minimal permission support for the absence of capital

Let the types of participants, goods, ports, technologies, and financial instruments be U, G, P_0, T, J, respectively. An original state x records stocks by participant and good, the complete claims matrix, technology, clock, and round. An original law L specifies request validity, actual debits and credits, port controllers, and admission rules. A request carries its signer, consenting parties, and operation. An execution word is a finite sequence of requests, each retaining the state it actually encounters. Rejected requests remain in the sequence; their actual material effects are determined by the original interpreter.

Write $a_L(t,p,u) \in \{0,1\}$ for the admission of participant u to port p under technology t, and $c_L(t,p)$ for that port's controller. Permission asymmetry occurs at an address where the controller is admitted and another participant is excluded:

$$\Phi_L(x,p,u) \iff a_L(x.T,p,c_L(x.T,p)) = 1 \ \wedge \ a_L(x.T,p,u) = 0.$$

Define

$$\mathrm{NoAsymmetry}(L) \iff \forall x,p,u, \quad a_L(x.T,p,c_L(x.T,p)) = 1 \Longrightarrow a_L(x.T,p,u) = 1.$$

The quantifiers range over all original states, ports, and participants. `Global.NoCapital` is the original five-part exclusion contract: the absence of this private phase, a capital-relation kernel, proper-title appropriation, actual-surplus appropriation, and fresh historical control.

Proposition B.1 (Exact characterization of minimal support). For arbitrary types as above and an original `RawLaw`, with the decidable equality structures on participants, goods, and financial instruments used by the original declaration,

$$\text{Global.NoCapital}(L) \quad \Longleftrightarrow \quad \text{NoAsymmetry}(L).$$

Opening every port to every participant is a sufficient special case. Uniformly closing every port also satisfies the same `NoCapital` contract. [CF1]

Proof outline. Boolean admission has only two values. The permission implication is therefore equivalent to the absence of Φ_L. The checked fields of an original capital-relation kernel contain this private phase. Proper-title and actual-surplus appropriation retain that dependency, while historical control carries the corresponding phase at its original occurrence. Thus `NoAsymmetry` excludes all five parts. In the reverse direction, the `private_phase` field of the complete contract excludes Φ_L at every address, recovering the permission implication. Under uniform closure its antecedent is false, giving the second exact instance.

The equivalence concerns the original law's complete exclusion property over all states. Observing that no capital kernel has occurred at one current state does not supply this premise over all states. Once the law satisfies the permission implication, however, the same exclusion proof applies at each state. The reverse direction requires neither an additional full-openness assumption, another price system, nor a successful production plan. The original registered root uses this permission characterization together with the existing responsibility, clearing, and actual-successor contract, giving the structural condition a definite role in subsequent execution.

This distinction determines the positive objective of public production. Uniform closure can remove asymmetric domination without generating subsistence payments or productive action. Public production in the main argument therefore requires capital exclusion, actual service payment, and an actionable successor together. An actual use of capital control also requires a valid original request invoking the relevant port: a permission label and a capital witness that has actually occurred are different objects.

Original declarations and direct uses (fixed version S). [CF1]: `the_existing_no_capital_contract_iff_its_minimal_permission_support` and `uniformly_closed_also_has_the_same_no_capital_contract` in `A/Quantifiers/Assembly/Strengthening/Characterization/Support/Law.lean`. The direct use is `the_original_registered_root_consumes_the_minimal_support_and_escape_characterization` in `A/Quantifiers/Assembly/Strengthening/Characterization/Consumers/Registered.lean`. It consumes the complete `MinimalSupportAndEscapeAt` of the original `seed/grossClaim` root and retains the previous clearing contract.

B.2. Unquoted discovery, renewed investment, and the current-period payment gate

An innovation mechanism M fixes its initial state, funding word, investment request, local policy, and closing word before the actual event arrives. A finite quotation grammar generates

`QuotedNature`; actual `Nature` accepts events from the original request type. The quotation domain supports prospective offers, while the actual ingress accepts the request that occurs. The policy reads local signals already acquired. A paid observation must leave a real positive debit. The original execution sequence retains the loan, material disbursement, investment, trial, observation, risk liability, and performance.

Proposition B.2 (Disposition of open events and two paid innovation rounds). For arbitrary original types, with the original decidable equality structures on participants, goods, and financial instruments, and any `Trials.Mechanism`, the public interpretation generates a complete disposition for every actual `Nature` and excludes a capital-relation kernel under that interpretation. For any first and second events, it generates either successful continuation in the second round or the original exact failure. The concrete original mechanism further generates the following two actual rounds. [CF2]

Quantity that actually occurs	First round	Second round
Acceptance result	Accepted	Accepted
Technology change	$0 \to 1$	$1 \to 2$
Actual investment, in material units	3	3
Total message-good consumption of participant 0	3	3
Subsistence and maintenance of every original participant	Paid	Paid
Debt from the original bearer to the old holder	Retained	1 at both the initial and final states

The second discovery, `discovery 2`, belongs to neither round's quoted menu. In the table, message is the original good's name: its total use includes production and observation. The original observation fee itself is one unit of message. At the start of the second round, the original experimenter's result stock is zero, yet the round acquires its own paid signal. Its quotation is recomputed from its own funded and paid prefix. The source starts at technology zero; finance, investment, and observation execute against the original stocks of each round.

Proof outline. The general conclusion classifies the complete execution sequence of the same interpreter. The public interpretation preserves the initial state, funding, investment, and policy, and excludes a capital kernel through open permissions. The checker returns the original failure object when funding, observation, safety, or continuation requirements are unmet. For the two concrete rounds, the original finite interpreter computes acceptance, technology, debits, claims, and subsistence and maintenance payment separately. Closing the first round consumes the tested results, preventing a retained prototype from serving as the second round's discovery. `nextTrial` replaces the mechanism's initial state with the first round's actual `next`, retaining the original law, participants, and program. Its new communication state has an empty packet list and the initial inbox, and its financing is read again at that actual successor.

Quotation and realized return also retain their respective sources. Let $p > 0$ be the declared success weight, L the weighted failure loss, a the outside option, and $b \geq 0$ an additional reward.

The original offer construction gives the success reward

$$q = \frac{L + a + b}{p}, \qquad \mathbb{E}_{\text{declared}}[\Delta] = a + b.$$

In the concrete source, the failure loss is 2, giving weighted loss 1 under equal weights. With $a = 0$ and $b = 1/2$, the reward is $q = 3$. Risk settlement selects the original risk issue and payment from the same actual inventory. Success pays cash 3; failure pays cash -2; the declared expected change is therefore $1/2$. Principal repayment, production income, and rent do not enter this risk-cash calculation. A finite contribution first becomes an original liability and is then performed with materials. A refused payment leaves both the risk claim and the old holder's claim visible.

Two controls make renewed payment explicit. Removing the second financing produces `unfunded`. Removing the second paid observation produces an `observation` failure and no `paidSignal`. In the latter branch technology may nevertheless have changed, separating a material discovery from the paid evidence acquired by the mechanism.

Original declarations and direct uses (fixed version S). [CF2]: `SourceInnovationLawAt` and `every_committed_source_generates_open_innovation_or_its_exact_original_failure` in `I/Completion/SourceLaw.lean`; `two_fresh_paid_trials_generate_successive_technologies`, `the_second_discovery_was_not_in_either_quoted_menu`, `the_next_trial_carries_the_old_debt_and_pays_again`, and `both_trials_pay_every_original_person_and_keep_their_reserves` in `I/Trials/Consumers/Renewal.lean`. [CF5]: `the_same_society_root_certifies_production_innovation_finite_contribution_and_control_loss` in `I/Completion/Assembly.lean` jointly uses the uniform innovation law, the two-round construction, original Engine boundaries, original financial liabilities, and the whole-ledger successor. `every_social_trial_attempt_keeps_the_original_claims_whole_ledger_and_generated_next` in `I/Completion/Society.lean` retains the original Society update for every original `oldVisit`, source index, boundary, starting index, actual event, and tick.

The original `nextTrial` supplies a template for the next trial. Releasing it requires the current period's subsistence and maintenance to have been paid. Let $W_M(e)$ be the word actually executed in the current round, and let $N_L(x, u)$ and $H_L(x, u)$ be participant u's necessary-subsistence and mandatory-maintenance bundles at the original initial state. `BundlePaid` means that the original word actually pays the original bundle for the specified bearer, goods, and purpose.

Proposition B.3 (Only current payment releases the next trial). For any mechanism M, first actual event e, and candidate next mechanism M', if

$$\text{paidNextTrial?}(M, e) = \text{some}(M'),$$

then $M' = \text{nextTrial}(M, e)$, the current trial has an acceptance witness, and its actual word $W_M(e)$ has paid $N_L(x, u)$ and $H_L(x, u)$ for every original participant u. An original counterexample has a future proposal satisfying the payment check and a current trial accepted by the previous trial check, while the word actually executed in the current period has not paid subsistence and maintenance. The gate therefore returns none. [CF3]

Proof outline. `prepareNextTrial` first reads the current `generate` result and then checks actual subsistence and maintenance participant by participant. A clear list check constructs `renewed`;

otherwise it returns a subsistence or maintenance failure carrying the original participant and an unpaidness proof. Eliminating the `some` branch exposes payment. The original `parties_cover` extends the list-membership conclusion to every original participant. In the counterexample, the feasible future word is not the current actual payment and cannot pass this check.

Original declarations and direct uses (fixed version S). [CF3]: `prepareNextTrial` and `every_released_next_trial_has_paid_the_actual_first_period` in `I/Trials/Source/Renewal/Decision.lean`. Their direct positive and negative uses are `the_original_paid_trial_passes_the_actual_period_renewal_gate` and `the_period_gate_keeps_the_unpaid_original_period_visible` in `I/Trials/Consumers/RenewalGate.lean`. The `ProductionInnovationContributionAndControlLawAt` contract under [CF5] retains both together with the same Society's actual whole ledger and `next`.

B.3. A cumulative receipt bound over every slot of an original contribution

The original interfaces in this section use decidable equality on participants, goods, ports, and financial instruments. The parameters `depth/count/prefixLimit/renewalLimit/sourceDepth/sourceCount` are arbitrary natural-number indices of the original declarations and remain attached to their source.

A finite contribution should have an executable settlement amount. Assignment and waiting must not reissue that same contribution. To state this requirement precisely, the original financial account retains an ordered list of obligation slots. Each slot stores its original claim, its issuance occurrence `origin`, and a remaining balance or terminal archive. Quantities use the claim's original unit; a change of holder does not change its source.

For an original account run $r : A \rightsquigarrow B$, let $p_i(A)$ be the remaining principal in original slot i, and $R_i(r)$ the sum of actual payments to that slot along the run. A new admission contributes zero actual payment, as does an assignment. A settlement reads the allocated quantity of its original payment event. Define cumulative receipts for one source over every closing slot by

$$R_o(r) = \sum_{\substack{i < |\,\mathrm{slots}(B)| \\ \mathrm{origin}(B_i) = o}} R_i(r).$$

The sum ranges over all live and terminal slots at the endpoint, rather than selecting a favorable position.

Proposition B.4 (Changing slots cannot regenerate the amount of an original source). Fix any original `ProgrammeSource.Input` and the natural-number parameters of the original declarations. For every finite continuation of its actual cursor, or every further finite public input list following the actual/public prefixes of its original registered account, let o be the issuance origin of any slot already present at the starting point and q_o its original issuance quantity. Then

$$0 \le R_o(r) \le q_o.$$

The same conclusion applies directly to the original innovation Engine, for every actual event, public stage, further original input list, and slot present at that starting point. If $d : \mathbb{N} \to \mathbb{Q}$ is

unbounded, meaning that for every rational bound b some k satisfies $d(k) > b$, these continuations of the same source cannot pay every $d(k)$. [CF4]

Proof outline. The first layer is conservation within each slot. An original event satisfies

$$p_i(\text{after}) + \text{paid}_i = p_i(\text{before}).$$

A new admission does not pay an old slot. Assignment preserves total principal, and an actual payment consumes only the original available allocation. Induction along the run telescopes these equalities:

$$p_i(B) + R_i(r) = p_i(A).$$

Generated accounts have nonnegative balances, while original admission binds principal to its own quantity. Hence $R_i(r) \le q_i$.

The second layer extends this local bound to every slot bearing the same source. Positions within an original inventory do not repeat; emitted claims form a sublist of the original requests. Every issue retains its original visit, and strict ordering between visits excludes the reappearance of that same issue. The original account therefore generates the property "at most one slot per origin" and retains it through every inventory prefix, actual cursor, and transport of the complete old account. Uniqueness is produced by the occurrence program, rather than supplied as an additional assumption.

The actual and public phases continue to retain their own original items. Occurrence readouts use the original phase and past length. Positions within a phase are strictly ordered, and the two phases have different tags. Each admission appends only its own original item origin, while old-account and new-epoch origins occupy the two branches of the original sum type. The complete actual/public account therefore also generates origin uniqueness. The original run retains the prefix of slot claims, so filtering the closing sum by the same origin leaves exactly the original slot:

$$R_o(r) = R_i(r) \le q_o.$$

This lifting retains every original live slot, terminal archive, and ordered history. Legacy transport follows the original generated run edge by edge, preserving origins through the injective mapping of the original constructor. It does not reset the account to an empty ledger. A payment quota is also carried only once by its original event; the next slot receives the quantity left after allocation to the preceding slot. A change of holder, passage through a public-law ingress, or continuation at another actual position therefore cannot bypass the original issuance quantity.

The identity splitting the public fold at its actual prefix attaches every further input list to this same account. The original innovation Engine uses the result directly through equality of its complete account.

An original control shows why this step is necessary: the generic native constructor permits two consecutive `admit claim` operations and really does duplicate the origin. Actual source folds exclude that reissuance. For unbounded d, choose a term larger than q_o; it contradicts the aggregate bound. This is a bound on cumulative receipts, not a date by which settlement must finish. A new actual service may have its own new origin and quantity.

Original declarations and direct uses (fixed version S). [CF4]: `an_original_slot_keeps_its_total_receipt_and_residual_exact` in `A/Contribution/Flow.lean`; `receivedForOrigin` and `the_actual_source_bounds_the_whole_original_contribution_receipt` in `Source/Receipts.lean`. The full-cursor use is `every_actual_cursor_continuation_bounds_the_whole_original_contribution` in `Source/Cursor.lean`, quantifying over `physical/start/later` and each starting slot. `every_actual_public_suffix_bounds_the_entire_original_contribution` in `Epoch/Consumer.lean` quantifies over `oldPast/past/further` and each starting slot. `every_original_innovation_public_suffix_bounds_the_whole_original_contribution` in `Epoch/Actual.lean` quantifies over the original innovation `nature/stage/further`. These relative paths start at `A/Contribution/`. Origin uniqueness is constructed in `Source/{Inventory,Issued,Accounts,Legacy}` and `Epoch/{Occurrences,Admissions,Accounts,Complete}`. The control is `two_free_admissions_really_do_duplicate_the_same_origin` in `Consumers/Admission.lean`.

B.4. Methods can be reused, and production can continue after exit

The public-production cell identifies participants by natural-number addresses and distinguishes resources through seven original goods. A `PaidMethod` retains the complete word actually audited, its original initial state and item, validity, and labor payment. A new plan is generated by retargeting the tail of that word item by item. Eligibility to use it is checked again against the real resources of the new current state. [PP1]

Write $x \oplus \delta$ for the complete state x with a nonnegative amount δ added only to the mother cell's labor stock. All other stocks, claims, technology, clock, and round remain unchanged.

Proposition B.5 (Preservation of complete capabilities and actual autonomous production). Under the original Cell law and its generated programs, the following results hold together.

(i) SharedAudit obtains its method after actually paying one unit of audit labor at original visit 4, and actually reuses it at visit 7. At frame 11, credits agree between the two executions for every natural-number participant and all seven goods; debits are lower only by one unit of mother-cell labor. The complete final states satisfy $x_{\text{shared}} = x_{\text{base}} \oplus 1$. Every finite word w that is valid item by item at the old endpoint remains valid item by item at the new endpoint, and

$$\text{execute}(x_{\text{shared}}, w) = \text{execute}(x_{\text{base}}, w) \oplus 1,$$

where execute denotes the complete final state. [PP1][PP2]

(ii) Participation incorporates joining, review, actual refusal for lack of resources, supply, exit, refusal of a role, autonomous production, and return into two actual 25-frame histories. At frame 25, the same flow relations over every address, complete-state difference, and transport of every old legal finite word still hold. At frame 21 one participant has exited and another has declined. Although the audit is valid and affordable, the emitted assignment word is empty. At frame 22 the exited participant actually produces and pays subsistence and maintenance, with a net increase of two seeds at the next frame. The participant subsequently returns by paying one unit of labor. [PP2][PP3]

(iii) For the same 25 frames of `Participation.source true`, Raw and Ledger each generate complete acceptance. Both yield a labor dividend of one, inventory at every address equal to the original endpoint, and termination of eight newborn obligations. The original root jointly consumes the original rows, whole ledger, and actual `next`. [PP4]

Proof outline. The method's private constructor is available only at original round 4 when the complete original word is valid. Its original item has an actual debit of one unit of mother-cell labor. The reused plan retains the parameterized structure of the audited program. Insufficient resources return blocked; an absent method returns to the previous program. Reuse of a method and provision of current materials are therefore checked separately.

Transport of an old word starts with a single step. Under the Cell law, additional nonnegative labor preserves the original request's validity conditions and resource inequalities. The original debit and credit operation commutes with this same labor increment. Induction over the finite word gives validity of the whole word and the complete final-state equality. Accounting over the entire address space first covers the actual finite support and then proves zero effects outside that support, extending the finite calculation to every natural-number participant. The 25-frame accounting includes all participation costs anew and uses the same single-step transport lemma. Its strict difference is still exactly one unit of labor.

Service preservation covers each actual intermediate cut. Before and after every actual item, the three original participants and cells already holding positive seed stock retain at least one unit of subsistence and maintenance goods. The complete claims matrix is recovered from that source's own native account, with the actual archive of the old principal retained. The labor saving therefore passes four checks together: flows, stocks, service floors, and original responsibilities. Resource-refused attempts remain recorded, and a payment completed in another history does not fill a gap in this one.

Exit changes standing through an original operation signed by the participant and actually accepted. The complete native history is

$$\text{admitted} \to \text{maintained} \to \text{deferred} \to \text{maintained} \to \text{bearerRelease} \to \text{maintained}.$$

A review refused for lack of resources leaves deferred status. Exit retains original admission and complete history; return continues along the original responsibility. Affordable work that is nevertheless not assigned, and actual personal production after exit, separately test refusal of a common role and retention of productive capability.

The two audits have definite algorithms. Raw scans original occurrences to match service payments and sums actual credits and debits. Ledger replays the original six service slots and the eight new slots generated by four births in this source, then folds inventory and costs in sequence. Newborn responsibilities are admitted at the original occurrence of receiving seeds; only the first valid payment within the original window creates the archive. The four births have distinct original positions: $0 \to 1$ at visit 4, $0 \to 2$ at visit 7, $1 \to 4$ at visit 9, and $2 \to 5$ at visit 10. Each generates subsistence 1 and maintenance 1 with a service window from its birth clock to that clock plus 3. Birth admission reads only the seed transfer that has already occurred; future payment is not an admission condition. Payment preserves that birth's original lineage and incidence.

The two routes share the original law, original occurrence, and generic lifecycle kernel. Each generates complete success without using the other's certificate as an input. The original root's two readout faces then recover this same occurrence and its actual successor.

Original declarations and direct uses (fixed version S). [PP1]: `PaidMethod.actual_labor_payment` and `PaidMethod.plan_is_parameterized_original` in `P/Cell/SharedAudit/Method.lean`; their direct use is `paid_schema_is_generated_in_the_original_root` in `Lean/SaturationMonoid/Tracks/LivingLawPublicCellDividendInstance.lean`. [PP2]: `every_legal_word_keeps_the_same_extra_labor` and `actual_dividend_preserves_every_old_executable_continuation` in `P/Cell/SharedAudit/Continuation.lean`; the complete 25-frame quantifiers are retained by `Certificate.old_capabilities` and `certificate` in `Participation/Runtime.lean`. [PP3]: `withdrawal_does_not_cancel_own_production` and `voluntary_return_is_an_actual_paid_choice` in `Participation/Effects.lean`, jointly used by `actual_paid_exit_public_production_and_strict_dividend_are_in_one_root` in `Tracks/LivingLawPublicCellParticipationInstance.lean`. [PP4]: `both_replays_accept_the_same_original_trace` and `every_new_source_birth_has_its_own_native_history` in `Tracks/LivingLawPublicArtifactZeroInstance.lean`. Participation paths in this paragraph start at `P/Cell/SharedAudit/`; Tracks paths start at `Lean/SaturationMonoid/`.

B.5. Resource units, subsequent payment, and occurrence identity

The quantities above answer different questions. An original contribution amount is measured in claim units, with actual materials recovered through the original instrument's unit and rate. Investment 3 and risk cash of 3 on success or -2 on failure come from their respective material entries in the innovation example. The Cell dividend of 1 is mother-cell labor. Equality of numerical values does not generate a common price. The original risk payment also retains the old debt of 1; finite compensation does not create permanent control. [CF4][CF5]

A separate, explicit `OpenReplication` cost family measures fuel. For every $n \in \mathbb{N}$, both first-window executions deliver service $n+1$ and maintenance 1. Exclusive fuel debit is $n+4$, public fuel debit is 3, and the difference is $n+1$. At subsequent visit 2, each execution computes $\lfloor \text{fuel} - 1 \rfloor$ from its own actual stock. The public execution actually spends the same saving on $n+1$ additional maintenance payments. Total fuel debit over the two windows is equal: the initial saving, temporary stock difference, and subsequent payment are successive uses of the same fuel. Under this family's zero-marginal-cost replication primitive, every later window additionally receives actual service.

This payment result is directly consumed at fixed S by `strict_dividend_and_its_actual_use_are_consumed_by_the_original_root` and `every_later_access_response_is_consumed_by_its_own_root` in `Tracks/LivingLawPublicDividendInstance.lean`. It uses a different original law and resource unit from the Cell's finite consumables and labor dividend.

The object preserved by each comparison must also be the same. Cell compares actual programs under one original law and a common initial state, proving an exact difference of complete final states. The innovation control comparison preserves the same seed, financing, investment, policy, and history-response function, and computes consequences from each actual execution. The two audits instead apply different algorithms to precisely the same history that has already occurred. The first two comparisons retain the responsibilities of two separate executions; the third proves

that both readouts recover a common carrier. These relationships determine where equality applies, which record pays a cost, and which actual `next` can continue.

A common starting point permits exact comparison, while full occurrence identity remains with each actual history. SharedAudit's 11 frames, Participation's 25 frames, the two innovation rounds, and OpenReplication each have their own source, payments, and `next`. The two innovation rounds are not part of the public cell's 25 frames. Raw and Ledger, by contrast, are two readings of those same 25 frames; their two acceptances are not two acts of production. Existing methods can be reused, while every new cell's materials, subsistence, and maintenance remain payable through its own actual payments. At the end of the original 25 frames, CopyReview reports real deficits for the subsequent fixed recipe. The complete acceptance of the preceding history and the next obligation to supply resources are thus retained together. [PP1][PP4]

Appendix C. Public Functions and Paid Planning

Continuity of public service and continuity of an officeholder are distinct institutional requirements. This appendix states the contracts that make the distinction executable: a replacement must inherit the original task, responsibilities, and actual accounts; authority must expire in the actual successor; and the planner's own work must enter payment and pause records. Propositions C.1–C.3 use current-frame at public-function source 6c245933. Proposition C.4 uses the independent planning source 59df7e39. The source register below identifies the precise declarations and their consumers.

C.1. Six contracts and every finite continuation

The original public-function input contains tasks, resources, bearers, local knowledge, actual control, and inherited responsibilities. Its domain includes states in which the current arrangement is captured and exit capacity is zero. A provider's standing is generated by actual events. Replacement receives the original requests, executes the original checks, preserves the particular rejected item on failure, and passes the original residual task into actual repair.[PF1]

Contract	Required output
Public source and actual selection	Original request, birth, and task identities; the complete report, funding, and service-window objects consumed after actual payment
Replacement and repair from zero exit	An actual replacement, or an exact rejection carrying its original position, item, and pending tail; the repair's own successor
Task homology and complete responsibility	Original service projections, individual responsibility identities and histories, financial prefixes, all received baselines, and the successor of the same execution
No legitimate final authority	Active responsibility does not confer permanently effective original authority on an incumbent
Complete net-capture bound	Three decisions jointly generated from actual payment in the same window, personally executable deviations, and complete costs and benefits

Contract	Required output
Default recursion and arbitrary rounds	The successor generates its next input, preserves these objects segment by segment, and supplies a complete disposition at every round

Proposition C.1 (Continuation on the same path, with complete dispositions). Fix any actual current state C_0 of this source, and define

$$C_{n+1} = \mathrm{target}(\mathrm{tick}(C_n)).$$

For every $s, k \in \mathbb{N}$, the original executor produces an actual evolution segment $E_{s,k}$ from C_s to C_{s+k}, with

$$C_{s+k} = \mathrm{causalAt}(C_s, k).$$

The segment preserves, jointly: the complete initial request directory as a prefix of the final directory; every received future baseline and its ancestry; each original responsibility's identity, complete history prefix, and native run from its original initial state; and the financial receipt for this same evolution. At every n, the next step produces an exact block, an actual repair, or a fully reviewed service disposition.[PF2]

The quantifiers cover every starting index and every finite segment length. They establish segmentability of a single source-generated path. Each tick takes its own input word from the source actually saved at that point, receives the old pending work and the new input, rebuilds the task, and invokes the original planner. The original executor's endpoint agrees pointwise with this recursion. Induction therefore carries the original run through each segment, rather than constructing a separate history with a similar endpoint.

Responsibility is preserved individually. If r is an original responsibility at C_s, its transported version r' satisfies

$$\mathrm{origin}(r') = \mathrm{origin}(r), \qquad \mathrm{history}(r) \preceq \mathrm{history}(r').$$

The responsibility's own run witness and the financial receipt travel along $E_{s,k}$. Rebuilding the task cannot reset identity, actual payment, or history. Directory extension preserves requests awaiting examination; baseline transport preserves opportunity commitments already received. These are distinct obligations.

Service projections retain the original bearer, quantity, purpose, and window. Checking each slot keeps an individual bearer's shortfall visible. The original rejection position, item, and pending tail enter the same repair. An incoming word that has already been received is not replayed merely because a new round begins. Different replacement paths retain their own financial runs and successors; multiple views of the same occurrence share its original carrier.

Complete disposition follows the original case split. An unrepaired rejection retains the original rejection row, task, and blocked status. A repaired branch returns the actual repair's `nextCurrent`. A service branch returns the original service, actual writeback, and complete review. Its internal cases distinguish an unavailable entry, ordinary completion or rejection, a short world budget, and completion of a world proposal. A short budget carries failure of the original cost-coverage check;

a rejection carries its original row; selection carries the acceptance proof. Every selected world candidate preserves the original queries, the current future, and ancestral futures, and uses the exact actual successor checked before selection. The tick after a block still has the original source's receiver.[PF2]

Replacement thus has a concrete institutional meaning: a proposal must inherit existing commitments before it can enter as an improvement. Failure retains the objects needed for further action. Whether service succeeds is determined by that round's actual resources, payments, and checks.

C.2. Original authority cannot revive along an actual suffix

Let ε denote the original authorization epoch and d_ε its deadline. Effective original jurisdiction at an observed state x is defined by

$$J_\varepsilon(i, x) \iff \mathrm{holder}(x) = i \ \wedge \ \mathrm{clock}(x) \leq d_\varepsilon.$$

An original seat comes from an actual grant signed by both parties; its effectiveness also preserves the original nominator's membership in the task. Task use must consume the corresponding authority. Discharge of responsibility is determined by its own termination receipt.[PF3]

Proposition C.2 (No final authority for the original epoch). Fix the original epoch and actual inventory. If the original settlement check returns false, then, for every actor i in this source and every actual evolution suffix starting at the original `Iteration.next`, its final observed state x satisfies

$$\neg J_\varepsilon(i, x).$$

Moreover, there is no public absorbing region that contains the current state, maintains active responsibility and the same actor's effective original jurisdiction at every member, and is closed under all accepted actual evolutions.[PF3]

The first claim rests on the original response actually producing a successor beyond the original deadline and on the clock never decreasing along actual evolution. The jurisdiction requirement that the clock remain at or before the deadline then contradicts the expiry preserved along every actual suffix. The conclusion covers arbitrary later actual paths.

The second claim has a directly checkable counterpath: submit empty words for the finite horizon generated by the original source. Every step is accepted, the original technology and incumbent remain, and the clock actually advances to expiry. If the proposed absorbing region existed, closure would retain that endpoint while still requiring the original authority to be effective. Responsibility identities and complete histories continue to be transported along the path; expiry preserves responsibility.

Effective jurisdiction and physical incumbency are read separately. The expiry path deliberately retains the incumbent, making the original deadline the basis of the loss of authority. At the same time, responsibility transport keeps the original bearer, occurrence, and history traceable. An organization can continue coordinating service; renewal of its original seat must be established by lawful events at that time.

The institutional principle is precise: **inherited responsibilities continue; expired authority must qualify anew.** Later cooperation may generate new tasks, deadlines, and authorizations. New permission requires new nomination, acceptance, and actual admission. Historical contributions, outstanding debts, and services still due retain their own places in the continuation.

C.3. The complete net-capture bound and three decisions

Fix an original bearer i, an actual baseline word b, the original continuation count, and a fixed price system p expressed in a common unit. The menu contains the personal service words generated for the two parties, the original first word, and the original method candidates. An available candidate a for i must be legal, with its signatures and consents under i's own control. A personal service word is generated from affordable production, actual supply to the original bearers, self-nomination, acceptance, and original service uses. Each compared path has its own complete ledger and successor.[PF4]

Let $M_i(b)$ be the original menu candidates passing the same-window check: both a and b are legal, every original service slot is actually fulfilled on each path, and their clocks at the original comparison cut are equal. Payment is established by their respective execution receipts. Under the fixed valuation, write $F_s(w)$ for the net financial burden attributed to the original selected lineage, $F_r(w)$ for its complete complement, and $V(w)$ for the valuation of original nonfinancial slots, complete events, and agency, less actual time and dynamically generated original production shortfalls. Then

$$U(w) = V(w) - F_s(w) - F_r(w), \qquad F_s(w) + F_r(w) = \sum_g p_g \left[Q_g^{\text{before}}(w) - Q_g^{\text{after}}(w) \right].$$

The second identity recovers the two components from the initial and final net positions of the same actual financial run.

Original production shortfalls are retained cell by cell as responsibilities are born. Each cell satisfies

$$\text{remaining} + \text{paid} = \text{quantity}.$$

Prior compensation therefore does not erase the original gross shortfall, and newly born shortfalls cannot be cancelled by treating the population as a fixed old cohort. Compensation's cash effect enters the actual financial ledger once, while the shortfall's valuation preserves its original occurrence. Original slots, history, agency, time, and financial consequences all enter the comparison.[PF4]

Proposition C.3 (Complete net bound under computed stability). For every $a \in M_i(b)$, the source generates signed cost entries $c_j(a;b)$ satisfying

$$\sum_j c_j(a;b) = F_r(a) - F_r(b) + V(b) - V(a).$$

The entries include the difference in the complementary financial account; valuations of the baseline's original slots and events; negative valuations of the corresponding candidate items; differences in agency and time; and the remaining and paid portions of each dynamic shortfall

cell on the two paths. Their signs retain the direction of each actual consequence. The complete complement makes the other side of a change in the selected net burden auditable.

Define

$$B(a;b)=\sum_j \max\{0,c_j(a;b)\},\qquad G(a;b)=\sum_j \max\{0,-c_j(a;b)\}.$$

If the computed list of strict improvements is empty,

$$\{a\in M_i(b):U(a)>U(b)\}=\emptyset,$$

then, for every $a\in M_i(b)$,

$$\boxed{F_s(b)-F_s(a)\le B(a;b)-G(a;b).}$$

Stability here is the actual computation on the original menu.[PF4]

The reasoning has two accounting steps. The entrywise identity $\max(0,c)-\max(0,-c)=c$ gives $B-G=\sum_j c_j$. The empty improvement list gives $U(a)\le U(b)$; rearranging the complete payoff definition yields the bound. All gains G are deducted, so the benefits of an actual improvement cannot disappear when its costs are aggregated.

The total decision exhausts three cases: the baseline's original service is unpaid; the source finds and returns a personally executable strict improvement for which both alternatives are actually paid in the same window; or the baseline is paid and the improvement list is empty, yielding the bound for every original menu candidate that passes the same-window check. Each branch retains its original evidence. The bound connects avoidable net capture to a specified bearer, window, genuinely available action, and complete consequences.

C.4. Comparison at the original cut, two invoices, and actual metering

Planning changes resource use and consumes resources of its own. The independent source 59df7e39 returns policy comparison to a shared cut before payment and preserves each candidate's own payment, execution, and recovery consequences.[PL1]

Fix a source-generated `Scope`, original reference horizon h, and post-payment horizon n, satisfying the original remaining-step bounds $h\le$ cursor.state.remaining and $n\le$ comparisonPoint.state.remaining. For an intent a and adaptive opponent ω, let $T(a,\omega)$ denote the complete tensor evaluated from the common original cut, including the present window, future restoration, and funding results. The generated joint-table set $\mathcal{T}$ covers both the old frame and each candidate's own post-payment frames; identical query keys share the same opponent response.

Proposition C.4 (Complete comparison and execution within the invoice). Within this original scope, for every binary relation R and any two original intents a,b,

$$[\forall t\in\mathcal{T},\ R(T(a,\omega_t),T(b,\omega_t))]\iff[\forall\omega,\ R(T(a,\omega),T(b,\omega))]\,.$$

A returned selection is nonregressive in complete responsibility among deployable intents and has no full-responsibility dominator. If a strict improvement exists in the original domain, the selection realizes a strict improvement. When no acceptable candidate exists, rejection retains a particular

failed row for every original intent. The actual installed full window is precisely the candidate's compared window, including its own actual payment prefix.[PL1]

The equivalence follows because every adaptive opponent restricts to a generated joint table. That table simultaneously preserves each candidate's own complete run, and therefore its whole tensor. Finite checks consequently cover the same opponent's adaptive responses along the different candidate paths. Nonregression is checked on the original present, future, and funding rows: present comparison retains the declared windows; future comparison preserves restoration capabilities individually; funding rows require a capability with no prior funding failure to remain free of such failure. Selection uses a finite encoding that preserves the original preorder, followed by an independent verifier's scan of all candidates. An already committed reservation for a candidate family also enters deployability: the unpaid reference remains available for comparison, while subsequent execution must respect payment already made; an invoiced candidate must pass its own actual-payment check.

Source-generated budgets and execution order then connect comparison to actual work. Default recovery first executes the construction invoice. On success, the original catalogue session generates the full candidate directory from the source menu, depth, and counts, preserving order and duplicates. If the original pending length is p and this execution's actual catalogue is $\mathcal{L}$, the units of the second, selection invoice are generated from that same catalogue:

$$u_{\mathrm{cmp}} = 1 + p + \sum_{w \in \mathcal{L}} (1 + p + 3|w|).$$

The rows and units are identified with the original repair catalogue and original comparison meter, respectively. After the second invoice is actually paid, search uses the same catalogue. Refusal of the first invoice retains its actual endpoint. Refusal of the second retains the generated catalogue, the first payment, and the actual successor of the second attempt.[PL2]

The chain advances through current sessions:

Layer	Production and consumption
Session	Stores the current cursor, actual work, and history; completion is generated only from that history
Micro	Advances one actual internal event at a time; internal work preserves the world, and a completed stage produces its corresponding original world transition
Restoration	Retains each restoration's original request, actual recovery, report, and funding checks; completion emits the result of that execution
Bank (result generation and caching)	Advances forecast, window, and restoration computations and caches their completed results; Bank.Payment assigns each actual work event to a position on an original successfully paid invoice

Once all originally required invoices have been paid and the work allowance is $u > 0$, the original metering interface Bank.Payment processes cache reading, joint-table generation, result generation and caching, quotation, work-capacity checking, and selection. Each stage uses only the

preceding stage's unused work allowance. If $e_0, \dots, e_5$ are the actual work counts of the six stages, then

$$\text{spent} = \sum_{j=0}^{5} e_j, \qquad \text{spent} + \text{unused} = u.$$

Every actual work event injects into a unit position on the same successful invoice. Distinct events within a phase occupy distinct positions, and positions from different phases are disjoint.[PL3]

A pause is a substantive result. If reading, table construction, restoration, quotation, capacity checking, or selection remains unfinished, the result retains the current cursor, paid work, remaining conditions, and its actual successor. Completion must consume evidence from the same session. A decreasing rank explains progress in the proof; the actual session advances from its current state. When the original capacity check succeeds, the established capacity theorem supplies completion of the corresponding selection. The service plan and the declared source events required to produce it thereby enter the economic account together.

C.5. Design interfaces for public verification and private autonomy

Scoped evidence, three-valued verification, and actual replacement of the auditor are new design contracts. Existing dual replay and revocation mechanisms provide foundations; Appendix D.2 specifies the complete implementation and privacy acceptance requirements.

C.6. Source register

The original proof epochs and consumers remain registered here. Actual H0 modules, complete types, and independent checking entries are collected in Appendix E and the machine-readable binding.

All source paths below are relative to the fixed Homework repository. Expand the path aliases; declarations belong to the indicated module namespaces.

- `P` = `Lean/SaturationMonoid/ProcessGame/Society/PublicFunction`.
- `H` = `P/Mandate/Financed/Continuing/Closure/Objection/Opening/ScopeRevision/Pending/Joint/Handoff`; `W` = `H/World`.
- `R` = `P/Distributed/Reaction/Native/Future/Recovery`.
- `S` = `P/Distributed/Reaction/Native/Online/Planning/Selection/Execution/Prepaid/Whole`.
- `Q` = `S/Observation/Domain/Policy/Budget/Escrow/Capacity/Selection/Counter/Automatic/Review/Enumeration/Joint/Accounting/Quotation/CrossFee/Choice/Computation/Query/Operand/Forecast`.

[PF1] Original occurrence and the six contracts. Fixed commit `6c24593364cd7763e3cdccf8f3654996a6dc8b5f`. `W/TaskSource/Iteration/Core.lean`: `Current.Incoming.produce`, `Incoming.Result.uses_the_actual_receiver`, `old_residual_precedes_the_new_word`, `every_received_word_is_registered`. `W/TaskSource/Iteration/Autonomous.lean`: `tick_receives_the_actual_own_word`, `tick_calls_the_original_planner`. Actual consumer: `Lean/scripts/verification/public-source-owned-iteration-consumer.lean`. The original acceptance identity of the complete six-contract package is fixed by `Lean/docs/handoffs/process-generated-society-active-route.md` and `Lean/docs/audits/public-source-final-acceptance.json` at that commit; current-frame is adopted.

[**PF2**] **Arbitrary segments and complete disposition.** The same 6c245933. `W/TaskSource/Iteration/Continuation.lean`: `exact_observed_prefix`, `successive_prefixes`, `segment`, `every_later_prefix_keeps_the_complete_directory`, `every_later_prefix_keeps_every_received_basis`, `every_later_prefix_keeps_original_responsibility`, `every_later_prefix_keeps_its_financial_history`, `Disposition`, `ServiceReview`, `dispose`, `inspect`, `selected_preserves_all_received_capabilities`, `even_a_block_has_the_next_source_receiver`. The original protection producers are `FutureProtection` and `QueryProtection` in `W/TaskInput/Work/Actual.lean`. The successor identity is `W/TaskSource/Service/Actual.lean:selected_next_is_the_prechecked_actual_next`. Direct consumer: `Lean/scripts/verification/public-unbounded-continuation-contract.lean`, with 17 contracts. The actual two-round consumer is the one in [PF1].

[**PF3**] **Original authorization epoch.** The same 6c245933. `W/Input/Challenge/Redress/Shared/Sovereignty.lean`: `EffectiveJurisdiction`, `NoFinalAuthorityCertificate`, `no_final_authority`, `no_final_admitted_authority`, `legal_expiry_is_an_all_accepted_actual_path`, `legal_expiry_preserves_raw_incumbency`, `legal_expiry_exhausts_the_original_jurisdiction`, `no_public_absorbing_sink`. Actual consumption occurs in that module's `generate` and in `W/TaskSource/Iteration/Autonomous.lean:no_original_jurisdiction_returns_in_any_actual_future`.

[**PF4**] **Complete net bound.** The adopted package is 6c245933; its acceptance register identifies the original rent-science epoch as `46179f5a71befa335f40ece16ca0b839b4ba835b`. `H/Registered/Loss/Appraisal.lean`: `the_financial_sum_is_the_original_actual_net_change`, `every_checked_window_has_its_own_actual_payment_cut_and_signature`, `every_native_cost_and_benefit_enters_the_same_payoff_difference`, `the_budget_minus_all_gains_is_the_exact_signed_cost`, `computed_stability_bounds_the_original_selected_net`, `the_source_produces_an_unpaid_baseline_or_actual_improvement_or_paid_cost_bound`. Actual consumer: `Lean/SaturationMonoid/Tracks/LivingLawPublicInWindowBurdenInstance.lean:publicSourceDecision`. Direct verification entry: `Lean/scripts/verification/public-final-rent-mouths.lean`. The scope is the original finite menu, original same-window check, and fixed prices.

[**PL1**] **Complete initial cut.** Independent commit `59df7e39d81e8698f3cf8507e64b93829bb1cac3`. `S/Source.lean`: `finite_relation_iff_every_same_adaptive_opponent`, `actual_installation_is_the_compared_full_initial_cut_window`. Actual consumption occurs in `S/Selection.lean` through `selection_compares_every_same_future_adaptive_opponent`, `selected_has_no_full_responsibility_dominator`, `every_available_real_improvement_is_realized`, and `independent_replay_accepts_selection`. Direct consumer: `Lean/scripts/verification/quota-comparison-consumer.lean:compared_branch_retains_its_own_complete_actual_initial_cut_window`. The selection entry states the post-payment bound as `n ≤ (paidPoint scope).state.remaining`; the original comparison entry states it as `n ≤ (comparisonPoint scope).state.remaining`. In `Whole/Prefix.lean`, `at_keeps_the_complete_raw_observation` preserves `state.remaining` along each original prefix. Since `Journal.point = pointFor start journal.words`, `comparisonPoint` has the same remaining-step value as its starting `paidPoint`, so the two original post-payment bounds coincide.

[**PL2**] **Source-generated catalogue, metering, and two invoices.** The same 59df7e39. `R/Metering/Selection/Catalogue.lean`: `Execution.rows_exact`, `Execution.units_exact`, `source_rows`, `source_units`. `R/Metering/Quoted.lean:stageProgram` successively consumes the actual first invoice, completed catalogue, and second invoice. `Session.move_appends`, `Session.completed_wo`

rk, and Session.conservation in R/Metering/Selection/Native.lean and Stage.lean preserve the work of that execution. Direct consumer: Lean/scripts/verification/cross-fee-catalogue-session-consumer.lean. Original paid-recovery consumer: Lean/scripts/verification/public-native-recovery-metering-consumer.lean.

[**PL3**] **Actual sessions to original invoices.** The same 59df7e39. Session.completed in R/Source/Session.lean generates completion from actual history. Cursor.move_core, Session.completed, and Session.completed_full_work in R/Source/Micro.lean preserve actual work at both granularities. Q/Bank/Payment.lean: generateFor, Decision.covered, Decision.conservation, Decision.phaseSlot, Decision.phases_disjoint, Session.accepted_capacity_completes, Result.next. The Restoration→Bank connection is adopted through the original session mechanism in Lean/docs/mechanisms/framework/authority/public-native-recovery-metering.md at this commit and the direct consumers above. Metering covers the declared original source events. Finer primitive internals, a complete automatically sufficient budget, and preservation across new outer costs belong to that branch's subsequent contracts.

Design status. C.5 follows the memorandum “Public Verification and Private Autonomy: Bounded Disclosure, Dual Replay, and Auditor Replacement” (revised 2026-11-02). Bounded disclosure, three-valued verification, and actual auditor handoff are new contracts awaiting implementation. Existing local verification, dual replay, and authorization foundations retain their respective scopes. These new interfaces are registered separately from the closed public-function contracts at 6c245933.

Appendix D. Implementation Contracts for Money and Bounded Verification

This appendix specifies the institutional designs used in the main paper, distinguishing reusable existing constructions from connections that remain to be implemented. It defines acceptance objects; it reports no completed monetary pilot or privacy system.

D.1. Generative money

Carrier. Each claim records its source of effect, issuance unit and quantity, applicable resource, current holder, cumulative actual payment, residual allowance, and unsettled responsibility. Basic participation allowances and contribution income have separate sources. Credit preserves the original risk bearer and the promised rules for maturity, restructuring, and loss allocation. Many people may use one public result; each additional scarce-resource claim must still state its own basis of issuance.

Issuance and discharge. Issuance must be triggered by an independently verified effect under rules fixed in advance. Copying, renaming, splitting identities, or transferring a claim must not issue the same original source again. Actual expenditure reduces the allowance and obtains the corresponding resource or disposition of responsibility in the same execution. A failed branch preserves costs already incurred and its own successor. Exchanges across resources disclose units, valuations, and minimum service constraints. A settled contribution remains settled under new aliases; new labor, maintenance, or actual adoption can generate a new source.

Minimum acceptance. A finite service trial must check at least that repeated registration adds no allowance; changes of holder preserve provenance and unsettled responsibilities; two concurrent expenditures cannot each use the same residual allowance; incomplete delivery is not recorded as settled capacity; failures and disputes can be traced to the original occurrence; and basic services and appeals remain available after exit. Source authentication, identity splitting, collusion, and challenges require their own attack and recovery scenarios. A data type's private constructors do not supply a security mechanism.

Existing foundations and new components. The actual clearing and credit of Appendix A, the original-source cumulative bound of B, and the event-by-event tickets and successor preservation of C provide reusable components. A dedicated common carrier, issuance and spending procedures, double-spending protection, and a complete challenge protocol remain to be connected. The original computation–effect pilot proposal supplies a design entry point. This paper reports no measured money supply, transaction throughput, or returns to actual participants.

D.2. Scoped evidence and three-valued verification

Original claim. The first verification object is one original service slot: was service actually delivered to its original bearer within the original window? Public authorization specifies the claim, window, evidence fields, purpose, and permitted recipients. Scope is fixed before the result is observed. Two verification paths independently replay actual payment and the responsibility lifecycle from permitted evidence, yielding three outcomes: verified, refuted by evidence, or insufficient evidence.

Verification requires sufficient original evidence of performance. Refutation requires actual material inconsistent with the claim. Other cases retain insufficient evidence. Refusing access permission and refuting a factual claim are distinct events. When evidence within scope does not decide the matter, the procedure may request duly authorized supplementary material or preserve an unresolved status with time-limited service protection.

Actual replacement of the auditor. After the original authority is revoked and a new auditor is nominated and accepted, only evidence that remains valid within the same permitted scope is handed over. The new auditor independently checks the original claim, preserves original responsibilities, bears the additional verification costs, and generates its own actual successor. Revocation constrains subsequent access and effects; information already learned is treated according to its actual state. Previous responsibility does not automatically revive authority.

Privacy acceptance. Define the permitted information P first. Under the same lawful strategy and permitted source inputs, compare two inhabited actual histories with different private coordinates. Verification within scope must succeed in both, while cumulative observations vary only as P permits. Each additional disclosure must enter this cumulative boundary. This is an observation contract awaiting implementation. Proof irrelevance and private constructors in a formal language do not themselves provide confidentiality.

The existing Raw/Ledger acceptance of the same 25 frames, source receipts, authorization, and revocation are reusable. Scoped disclosure, three-valued conclusions, and actual auditor handoff form a new complete acceptance unit. The earlier success of dual audit does not certify this unit.

D.3. Social standing and technical objects

Bearers, roles, and new responsibilities in formal constructions have explicit sources. The principle of basic participation for persons rests on arrival, need, and being affected; it cannot be generated by a ranking of computational roles. Corporations, institutions, theories, and artificial systems can enter an analysis of responsibility. A technical object's continued existence or processing capacity does not automatically grant it coercive authority over persons.

Formal progress budgets, financial claims, material stocks, labor time, and ecological thresholds likewise retain their units. Institutional applications should first specify the correspondences among them before making quantitative claims. The eight principles of the main paper require these correspondences to have public reasons, actual costs, and contestable boundaries of power.

Appendix E. Formal-Proof Source Snapshot and Acceptance

E.1. Fixed snapshot

The formal proof materials use H0mework, commit `2d6fbc96e33b7c4c3cfd91104fda74e3caeda045`. Paths below are relative to that repository; snapshot availability is stated in the source note. The original e60, df, 1d, 6c, and 59df proof epochs retain their separate identities. Admission to one repository does not combine their original certifications into a single execution.

The source-key register retains 23 keys, 90 interface rows, 76 original epoch/file identities, and 172 named adoptions. The machine-readable binding supplies each full qualified name, actual public module, original declaration header, exact source alias, recursive import environment, and complete-type receipt location. Types are stored as complete Lean expression DAGs and checked for kernel equivalence.

E.2. Five main packages and eleven independent consumers

The five main module names concatenate the prefix `H0mework.Papers.EconomicConstitution` with the suffix in the table.

Material	Original source	Module suffix
Exit and common clearing	`e60a8605`	`E60`
Society, capital, public production	`df421ff6`	`SourceDF`
Original paid progress	`1d7db207`	`PaidProgress`
Public-function current-frame	`6c245933`	`PublicFunction`
Independent public planning	`59df7e39`	`Planning`

The original consumer programs keep independent environments. Their actual module names concatenate `HOmework.Versions.`, the public version, `.ReleaseMaterials.Scripts.Verification.`, and the module tail below. Each is a direct Lean entry; these programs are not coimported into the five main packages.

Public version	Independent entry tail	Original anonymous cases
R59df7e39	CrossFeeCatalogueSessionConsumer	58
R59df7e39	CrossFeeCatalogueSessionMutation	26
R59df7e39	CrossFeeCatalogueSourceConsumer	18
R59df7e39	PublicNativeRecoveryMeteringConsumer	4
R59df7e39	PublicWorkloadRecoveryConsumer	3
R6c245933	PublicCurrentInputContract	20
R6c245933	PublicDefaultRawReentryContract	18
R6c245933	PublicRecursiveTaskContract	17
R6c245933	PublicSourceOwnedIterationContract	8
R6c245933	PublicTaskReentryContract	12
R6c245933	PublicUnboundedContinuationContract	17

All 201 original anonymous cases have unique checking observation names. Their original ordinals, complete types, and values are preserved, and the original bytes are reversibly recoverable. An originally anonymous example is registered as such; no original qualified name is invented. The `consumer_programs` section of the machine binding records every observation name, original ordinal, and type index.

The original two-round actual program, original certifications, and these contract programs are registered separately. The original evidence retains the identities of 1200 source files and 58 receipts. The current sixteen-package acceptance covers 330 roots, 3313 complete declaration types, and all 201 observations. Original successes, failures, and unadopted dispositions retain their recorded verdicts.

E.3. Reproduction and checking entries

Material	Repository-relative path
Claim map	`docs/economic-constitution-map.json`
Finite selection	`checks/economic-constitution-selection.json`
Reproduction commands	`docs/economic-constitution-reproduction.md`
Acceptance note	`docs/economic-constitution-readiness.md`

Under `evidence/economic-constitution/acceptance/initial-intake/`, `summary.json` records the accepted scope. Files `build/result.json.gz`, `trust/result.json.gz`, and `types/result.jso n.gz` record the complete build, metadata-dependency audit, and full types. `publication.json` binds the executed map and logs; `cache/current-report.json` records cache identity. All sixteen packages passed locally with stable inputs, axioms `propext`, `Classical.choice`, `Quot.sound`, and zero `unsafe`/`partial`. Type references retain the receipt hash, declaration index, and type hash.